

雲端運算服務作業 (Azure容器應用)

系級	資料四B	學號	10173214	姓名	何書維
----	------	----	----------	----	-----

相關操作說明:

1. 建立翻譯工具服務

先利用下面的指令下載[翻譯工具的github](#)

```
git clone https://github.com/coachsu-azure/azure-translator
```

接著創建conda 虛擬環境，並安裝相關套件

```
conda create -n cloud python==3.10.13 # 雖然github選擇的是3.14版本，但我認為這個版本在安裝大部分  
pip install -r requirement.txt
```

接著到Microsoft Azure建立資源群組及翻譯工具

建立完之後，到翻譯工具下找尋金鑰，將其與地區的值複製貼入道專案中的./env檔案對應的參數，並利用下面的指令執行程式

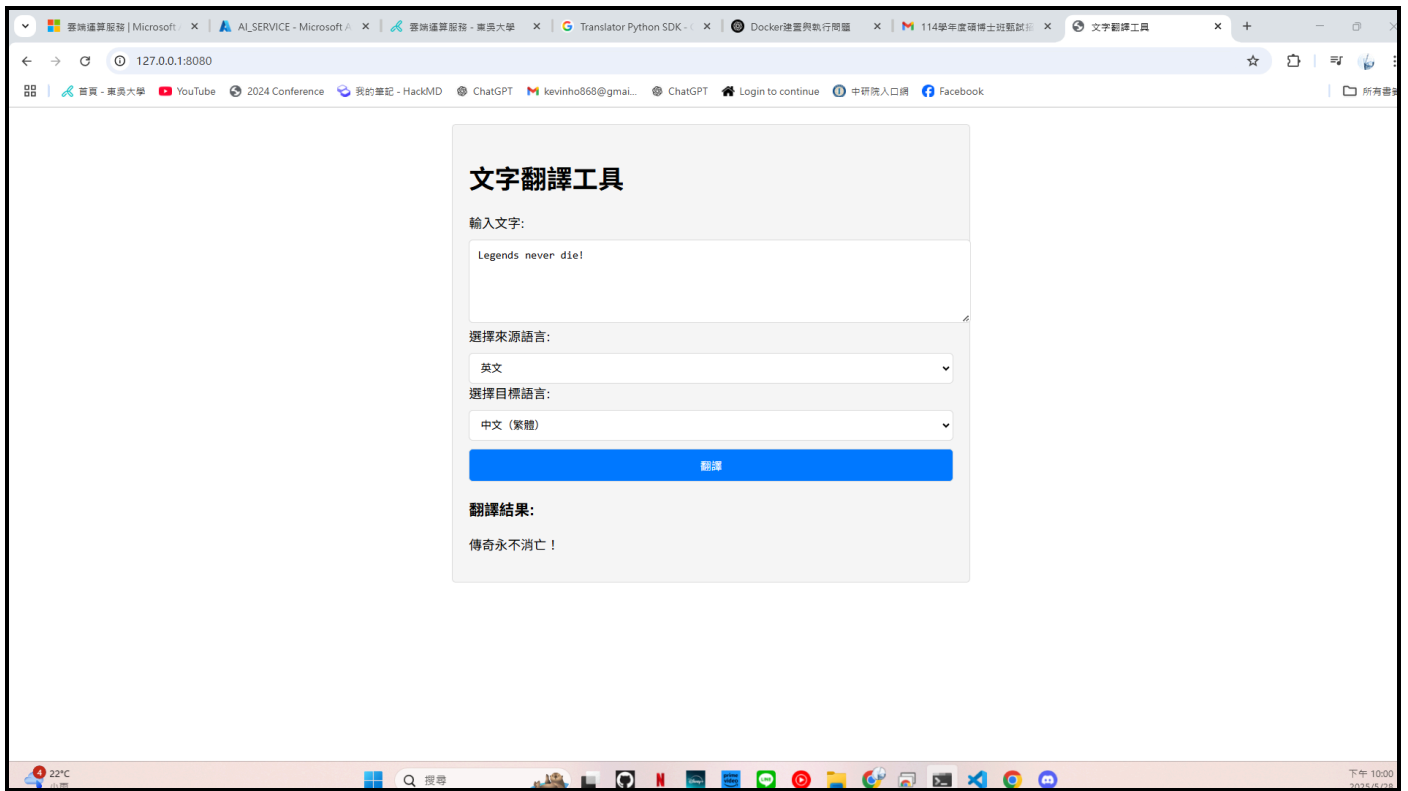
```
python web.py
```

就會如下圖所示

```
Windows PowerShell
(c:\cloud) PS C:\Users\User\Desktop\SCU_HW\雲端運算\azure-translator> python .\web.py
python : 無法辨識 'python' 詞彙是否為 Cmdlet、函數、指令檔或可執行程式的名稱。請檢查名稱拼字是否正確，如果包含路徑的話，請確認路徑是否正確，然後再試一次。
位於 線路:1 字元:1
+ python .\web.py
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (python:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

(c:\cloud) PS C:\Users\User\Desktop\SCU_HW\雲端運算\azure-translator> python .\web.py
southcentralus 6509Y9nWkMmVYfSnR2HuK4qn2uW4TFBnr74Pk8VkytltLJZR9X1wJQQ399BEACLArgHXJ3w3AAAbACOGg1Q0
* Serving Flask app 'web'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://192.168.1.104:8080
Press CTRL+C to quit
127.0.0.1 - - [28/May/2025 22:00:23] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [28/May/2025 22:00:39] "POST / HTTP/1.1" 200 -
```

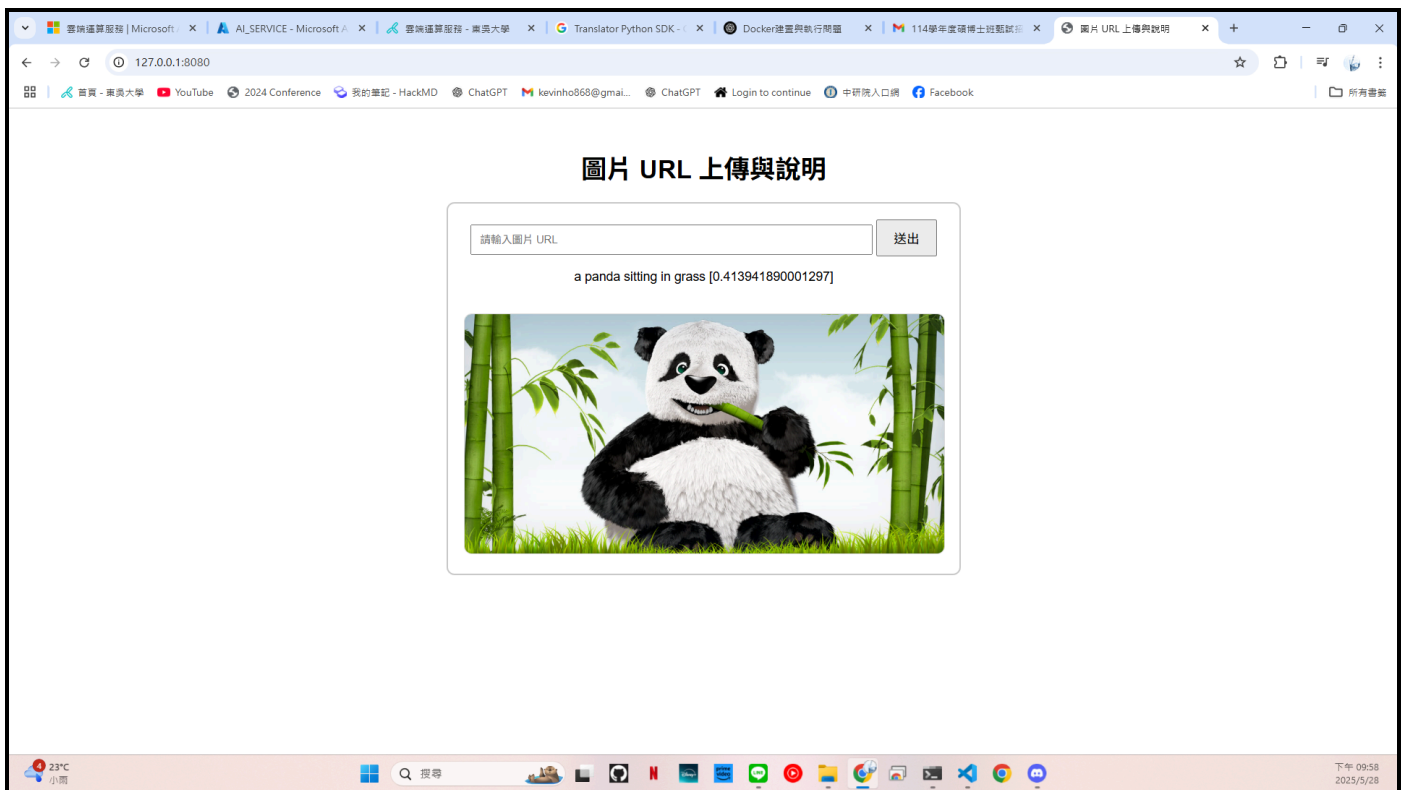
點開(<http://127.0.0.1:8080>)的連結之後，輸入自身想翻譯的文字並點選翻譯即可出現



2. 建立影像分析服務

如同上面的指令，將github換成computer vision，丟入網路上一張熊貓的圖片連結，點選送出即可回傳英文的描述

```
Windows PowerShell
Using cached urllib3-2.4.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: six>=1.11.0 in c:\users\user\appdata\roaming\python\python310\site-packages (from azure-core>=1.24.0->msrest>=0.6.21->azure-cognitiveservices-vision-computervision->r.\requirements.txt (line 2)) (1.16.0)
Collecting typing-extensions>=4.6.0 (from azure-core>=1.24.0->msrest>=0.6.21->azure-cognitiveservices-vision-computervision->r.\requirements.txt (line 2))
Using cached typing_extensions-4.13.2-py3-none-any.whl.metadata (3.0 kB)
Collecting oauthlib>=3.0.0 (from requests-oauthlib>=0.5.0->msrest>=0.6.21->azure-cognitiveservices-vision-computervision->r.\requirements.txt (line 2))
Using cached oauthlib-3.2.2-py3-none-any.whl.metadata (7.5 kB)
Using cached python-dotenv-1.1.0-py3-none-any.whl (20 kB)
Using cached azure_cognitiveservices_vision_computervision-0.9.1-py2.py3-none-any.whl (36 kB)
Using cached azure_common-1.1.28-py2.py3-none-any.whl (14 kB)
Using cached flask-3.1.1-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.2.1-py3-none-any.whl (102 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached MarkupSafe-3.0.2-cp310-cp310-win_amd64.whl (15 kB)
Using cached msrest-0.7.1-py3-none-any.whl (85 kB)
Using cached requests-2.32.3-py3-none-any.whl (64 kB)
Using cached charset_normalizer-3.4.2-cp310-cp310-win_amd64.whl (105 kB)
Using cached idna-3.10-py3-none-any.whl (70 kB)
Using cached urllib3-2.4.0-py3-none-any.whl (128 kB)
Using cached azure_core-1.34.0-py3-none-any.whl (207 kB)
Using cached certifi-2025.4.26-py3-none-any.whl (159 kB)
Using cached isodate-0.7.2-py3-none-any.whl (22 kB)
Using cached requests_oauthlib-2.0.0-py2.py3-none-any.whl (24 kB)
Using cached oauthlib-3.2.2-py3-none-any.whl (151 kB)
Using cached typing_extensions-4.13.2-py3-none-any.whl (45 kB)
Using cached werkzeug-3.1.3-py3-none-any.whl (224 kB)
Installing collected packages: azure-common, urllib3, typing-extensions, python-dotenv, oauthlib, markupsafe, itsdangerous, isodate, idna, click, charset-normalizer, certifi, blinker, werkzeug, requests, jinja2, requests-oauthlib, flask, azure-core, msrest, azure-cognitiveservices-vision-computervision
Successfully installed azure-cognitiveservices-vision-computervision-0.9.1 azure-common-1.1.28 azure-core-1.34.0 blinker-1.9.0 certifi-2025.4.26 charset-normalizer-3.4.2 click-8.2.1 flask-3.1.1 idna-3.10 isodate-0.7.2 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.2 msrest-0.7.1 oauthlib-3.2.2 python-dotenv-1.1.0 requests-2.32.3 requests-oauthlib-2.0.0 typing-extensions-4.13.2 urllib3-2.4.0 werkzeug-3.1.3
(ccloud) PS C:\Users\User\Desktop\SCU_HW\雲端運算\azure-computer-vision> python .\web.py
* Serving Flask app 'web'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://192.168.1.104:8080
Press CTRL+C to quit
127.0.0.1 - - [28/May/2025 21:58:36] "GET / HTTP/1.1" 200 -
https://tinypng.com/images/social/website.jpg
a panda sitting in grass [0.413941890001297]
127.0.0.1 - - [28/May/2025 21:58:41] "POST / HTTP/1.1" 200 -
```



3. 服務串接&容器化

由於最後呈現會是在影像分析的部分，因此將影像分析的web.py做一些修改如下所示:

```
"""
```

```
Azure AI 電腦視覺(網頁版)
```

```
"""
```

```
import os
```

```
import pathlib
```

```
from dotenv import load_dotenv
```

```
from flask import Flask, request, render_template
```

```
from azure.cognitiveservices.vision.computervision import ComputerVisionClient
```

```
from msrest.authentication import CognitiveServicesCredentials
```

```
from azure.ai.translation.text import TextTranslationClient, TranslatorCredential
```

```
from azure.ai.translation.text.models import InputTextItem
```

```
# 如果.env存在，讀取.env檔案
```

```
env_path = pathlib.Path(".env")
```

```
if env_path.exists():
```

```
    load_dotenv(dotenv_path=env_path, override=True)
```

```
# 取得環境變數
```

```
KEY = os.getenv("KEY")
```

```
ENDPOINT = os.getenv("ENDPOINT")
```

```
## 翻譯工具
```

```
TRANS_KEY = os.getenv("TRANS_KEY")
```

```
TRANS_ENDPOINT = os.getenv("TRANS_ENDPOINT")
```

```
TRANS_REGION = os.getenv("TRANS_REGION")
```

```
text_translator = TextTranslationClient(
```

```
    endpoint=TRANS_ENDPOINT, credential=TranslatorCredential(TRANS_KEY, TRANS_REGION)
```

```
)
```

```
SRC = "en"
```

```
DST = ["zh-hant"]
```

```
app = Flask(__name__)
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def object_detection():
```

```
    """
```

```
    Azure AI 電腦視覺(網頁版)
```

```

"""
image_url = ""
language = "en"
description = ""

if request.method == "POST":
    image_url = request.form.get("image_url")

    print(image_url)

    client = ComputerVisionClient(
        endpoint=ENDPOINT, credentials=CognitiveServicesCredentials(KEY)
    )

    analysis = client.describe_image(
        url=image_url, max_descriptions=1, language=language
    )
    result = analysis.captions[0]
    # 將回傳值進行翻譯
    translate = [InputTextItem(text=result.text)]
    responses = text_translator.translate(
        content=translate, to=DST, from_parameter=SRC
    )
    description = f"{responses[0].translations[0].text} [{result.confidence}]"

    print(description)

return render_template("index.html", image_url=image_url, description=description)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

除了程式碼加入翻譯工具套件以外，因為要連接翻譯工具的金鑰、端點以及地區，因此也在影像分析的./env中加入如程式碼中的相應參數

接著在本機端上執行下面的指令，看是否能夠成功執行

```

docker image build -t vision:latest .
docker run --name vision -p 8080:8080 vision:latest

```

執行成功畫面會如下圖所示:

```

- FromPlatformFlagConstDisallowed: FROM --platform flag should not use constant value "linux/amd64" (line 1)
(ccloud) PS C:\Users\User\Desktop\SCU_HW\雲端運算\azure-computer-vision> docker run --name vision -p 8080:8080 vision:latest
* Serving Flask app 'web'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.17.0.2:8080
Press CTRL+C to quit
172.17.0.1 - - [28/May/2025 16:17:47] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [28/May/2025 16:17:52] "POST / HTTP/1.1" 200 -
https://tinypng.com/images/social/website.jpg
一隻熊貓坐在草地上 [0.4139421284198761]

```

接著回到Azure中，建立Container registry，建立能夠透過docker 登入到Azure的帳戶，隨後找尋帳戶並在終端機上輸入以下指令

```

docker login devotion.azurecr.io
docker image build -t devotion.azurecr.io/vision:latest .
docker image push devotion.azurecr.io/vision:latest

```

來將映像檔導入Azure的Container中，接著建立容器執行個體並將網路的TCP port改成8080，建立之後利用容器執行個體產生的IP加上:8080到網頁上，即可跳出影像分析的網站，將上面丟過的圖片再丟入並送出，即可產生翻譯成中文的描述



心得

這次實作與上次較為不同，上次只需專注在雲端不同虛擬機的連線確認連線即可，但這次作業作為期末專題的前哨，比起上次只要連線成功即可這次多了更多實際的作用，而我本次作業遇到最大的困難點也出現在將翻譯工具以及圖片辨識進行串接，修改了本地端的程式碼數次，不斷將測試結果PUSH上去，都快將大部分地區的定價層免費額度都用光了，所幸最後還是順利完成作業，成功在額度使用完之前做出結果，讓我鬆了一大口氣。